

Technical Assessment (AI Track)

Assessment Overview

Select a problem relevant to Singapore public-sector stakeholders, such as a government agency, ministry, or citizen group. Develop an AI-powered prototype to address the identified problem, perform a comprehensive evaluation, and provide a Dockerized solution accompanied by professional documentation.

This constitutes the scope of the assignment. The following sections outline the criteria for a successful submission.

Assessment Rationale

This open-ended format is designed to evaluate critical professional competencies: problem identification, project scoping, and the exercise of technical judgment when utilizing AI tools. We seek candidates capable of determining project viability and performing rigorous output validation, as these represent the primary differentiators in technical expertise.

Technical Themes

AI must be integrated into the core functionality of the prototype. We have identified four primary areas of public-sector value:

1. **Unstructured to structured.** Pulling usable signals out of documents, images, audio, or video.
2. **Retrieval & knowledge.** Helping someone find, connect, or reason over information they could not efficiently search before.
3. **Generation with guardrails.** Producing content (text, slides, diagrams, synthetic data) that is grounded, useful, and safe for a public-sector context.
4. **Agents & workflows.** Multi-step systems that chain tools or actions to accomplish a defined job.

Candidates may choose one area, combine multiple themes, or propose an alternative application. Classification, detection, retrieval, multimodal systems, and agentic workflows are all acceptable methodologies.

Refer to Annex A for illustrative problem sketches. These examples are provided to indicate project scope rather than to serve as an exhaustive list of approved topics.

Submission Requirements

- Provide comprehensive instructions to enable the full deployment and execution of the application.
- The solution must demonstrate significant AI integration, rather than simply utilizing an API call within a standard web application.
- Select and justify a specific evaluation methodology (e.g., benchmarking, LLM-as-judge, human evaluation, ablation, or comparison). Provide a clear rationale for the chosen method in the README. The soundness of the evaluation methodology is prioritized over the magnitude of the results.
- Detail the provenance, creation or processing methods, licensing, and privacy considerations for all data used. There is a wide variety of available data (e.g., data.gov.sg, Hugging Face, Kaggle, agency publications, or hand-curated sets). Resourcefulness in identifying relevant data is a key evaluation metric. Ensure compliance with source licensing and robots.txt. If the data contains personal information, consider the ethical implications and appropriate handling procedures. Synthetic data is acceptable if it is generated meaningfully and with high fidelity; please provide a defense for its use.
- This is the primary artifact. Document the objectives, rationale, implementation, results, execution instructions, and limitations.
- Provide a concise section (150-250 words) in the README outlining the target user and environment, projected inference costs or compute footprint at scale, key performance monitoring metrics post-deployment, and an assessment of a specific potential deployment risk. We are not asking for the implementation of these considerations.
- Provide a brief narrative detailing the development process, including iterations, discarded strategies, tool selection rationale, and decision-making justification.
- Submit a 3 to 5-minute video demonstration (e.g., via Loom or screen recording) showing the application in operation.

Timeline and Submission Process

The submission deadline is five (5) days from receipt. We estimate a high-quality submission requires approximately 12–16 hours of focused effort; the extended window is provided to accommodate your existing commitments. The time of submission is not a factor in the evaluation.

Submission Protocol: Provide a private GitHub repository link and invite @govsg-ds-hiring as a collaborator. The repository must remain anonymous regarding the specific organization or role. Include the demo video URL in the README file. The repository must remain unchanged throughout the evaluation period. Submit the URL via the recruitment portal as per HR instructions.

Evaluation Criteria

Dimension	Weight	What we are looking for
Problem framing & creativity	20%	Is this worth solving? Is the angle interesting or non-obvious? Or does it look effort-shaped, the first thing that came to mind, phoned in?
Technical execution	25%	Does it run from your README on a clean machine? Is the code quality up to standard? AI-assisted coding is expected; careless output is not.
Evaluation & effectiveness	30%	Sound methodology. Novelty counts only together with effectiveness. Novel-but-underperforms does not beat tried-and-tested-and-works. Honest about limitations.
Data thinking	15%	Provenance, licensing, representativeness. Resourceful about sourcing. Thoughtful about what the data does and does not cover.
Communication	10%	Your writeup accurately describes what the code actually does. No overclaiming, no features in the README that are not in the repo.

Critical Non-Compliance Indicators

- **Lack of Reproducibility.** Failure to execute the solution following README instructions.
- **Code-writeup mismatch.** The writeup describes features, architecture, or results that do not match what is in the repo.
- **Insufficient Technical Justification.** Candidates must be able to articulate the technical rationale behind their code and architecture during subsequent interviews.
- **Unsubstantiated Model Selection.** Model choices must be justified by operational requirements such as data privacy, latency, or specific task suitability.
- **Substandard Code Quality.** While AI-assisted coding is permitted, the final output must meet professional standards for code quality and maintainability.
- **Recycling a Past Project.** Candidates must start a fresh project. If we have reasonable doubt of you recycling your capstone project (for example), you may be disqualified without justification.

Approved Methodologies and Resources

- **Tool Selection.** Candidates may utilize any accessible AI tools, including open-source weights and local inference. Organization-provided API keys or compute resources will not be issued. Submissions should clearly state any technical limitations and describe potential enhancements if those constraints were removed.
- **Resource Accessibility.** Solutions should be functional within free-tier resource limits. If the prototype requires paid infrastructure for production scale, this should be noted in the documentation.
- **Usage of AI and coding agents (Claude Code, Cursor, Copilot, Codex, etc) is permitted.** What we evaluate is the final artifact and your ability to explain the decisions in it. **If you used coding agents, please include a section at the end of your README on how you have used them to support your work.** In addition, upload all coding agent session and chat logs (e.g. jsonl from ~/.claude/projects/) to your repository and indicate their location in your README.
- **Local vs API models does not matter.** What matters is that you can justify the choice on grounds that hold up (data sensitivity, latency, task fit, fine-tuning needs, cost-at-scale economics). Not on grounds that do not (free, cool, had it lying around).
- **Free resources if you are starting from scratch.** LLM APIs: Google AI Studio, Groq, OpenRouter, Hugging Face Inference Providers, Cerebras, Together AI. Local inference: Ollama, llama.cpp, LM Studio. GPU compute: Google Colab, Kaggle Notebooks, Lightning AI. Open weights and datasets: Hugging Face, Kaggle.

Supplemental Guidance

- **Annex A** (below): around 21 problem statement sketches drawn from real public-sector work, anonymised. Pick one, remix one, or ignore them all, completely up to you. They are here to calibrate the shape of the space, not to constrain you.
- **Annex B:** [{build} 2025 public projects](#). Past GovTech hackathon submissions. Not all AI-focused, but a good sense of what real shipped prototypes look like in this environment, given a month-long window.

Annex A. Example problem sketches

Paraphrased and anonymised. They are here to show you the shape of the space, not to restrict you.

Vision, video, and multimodal understanding

1. **Property condition assessment from imagery.** A government tax agency assesses the construction era and condition of residential properties for valuation. Today officers eyeball street-level imagery, which is subjective, slow, and inconsistent across valuers. Could a vision model compress turnaround and improve consistency?
2. **Structured attribute extraction from animal photos.** A community ecosystem management programme needs to extract attributes from photos of stray animals, such as sterilisation status (via ear-tipping), gender, and approximate age. Could a vision model do this reliably enough to support routine operations?
3. **Urban disamenity detection from imagery.** Municipal issues like illegal dumping, overgrown greenery, or damaged infrastructure can be flagged from images submitted by officers or citizens. Could a VLM produce structured outputs (category, severity, location cues) from photos, with a real precision/recall eval?

Retrieval & knowledge

4. **Precedent-aware response drafting.** Officers have tight deadlines to draft responses to legislative or ministerial questions. The work requires precedent from past replies, press releases, speeches, and policy documents scattered across repositories. How might you compress the research phase so drafting gets the focus?
5. **Benefits matching for citizens.** Across dozens of government schemes, citizens may be eligible for many they do not know about. Existing portals are searchable but passive: the citizen does the discovery. Could a profile-driven matcher proactively surface entitlements?
6. **Legal evidence navigation.** Court bundles can run thousands of pages. Judges and legal teams need to find specific facts, arguments, and inconsistencies. Could you build something that lets them query evidence without hallucinating claims that are not in the record?

Generation with guardrails

7. **Document-to-presentation.** A common public-sector workflow: take a long policy document, produce a presentation for leadership. Could an AI system produce editable slides that match organisational conventions and actually communicate the substance?
8. **Targeted practice questions.** Students and parents want practice questions aligned to syllabus and specific topics. Existing options are generic or expensive. Could you generate practice that is both aligned and pedagogically sound, with an eval for correctness and difficulty calibration?
9. **Synthetic data for data-scarce vision tasks.** Some detection tasks lack enough real training data, such as oil spills, rare safety violations, or culturally specific artefacts. Could you generate synthetic visual data and show empirically that models pre-conditioned on it transfer to real data?

Agents & multi-step workflows

10. **Feedback-to-ticket pipeline.** Product teams receive user feedback. Humans read it, group it, write specs, brief engineers. Could an agent system handle the first mile (triaging, clustering, drafting tickets) without losing signal?
11. **Automated safety testing for LLM apps.** Given an LLM application endpoint, an agent generates adversarial prompts, probes for jailbreaks, and produces a structured safety report. Signal is in how you generate good adversarial prompts, not the plumbing.
12. **Triage agent for citizen emails.** Agencies receive high volumes of emails covering many use cases against complex internal SOPs. Could an agent read the email, classify it, look up the relevant SOP, and draft a response, with calibrated confidence and clean human escalation?

Evaluation, benchmarking, robustness

13. **AI output quality harness.** AI-generated content is flooding workflows and quality is inconsistent. How might you build a lightweight eval harness that catches low-quality AI output before it reaches readers, without being so strict it blocks good output?
14. **Localised safety benchmark.** Existing LLM safety benchmarks are mostly US-centric. Could you construct and validate a small benchmark that tests for Singapore-specific context (Singlish, local slang, local references) and use it to evaluate at least two off-the-shelf models honestly?
15. **Out-of-knowledge-base robustness.** RAG systems often confabulate when asked questions their knowledge base cannot answer. Could you build a small benchmark and eval tool that measures how often a RAG system acknowledges “I do not know” vs. makes things up?

Annex B. {build} 2025 public projects

<https://build.tech.gov.sg/projects/2025>. Browse for a sense of what past GovTech hackathon submissions look like. Not all AI-focused, but a useful calibration for what “reasonable scope shipped in a short window” means in this environment.